

🔗 INTERACTIVE SQL PART – 1

Table fundamentals :

A table is a database object that holds user data.

⇒ Oracle datatypes

CHAR, VARCHAR/VARCHAR2, DATE, NUMBER, LONG, LONG ROW

✓ *CREATE TABLE Statement*

```
CREATE TABLE <table_name>
(
  column_name1 data_type(size),
  column_name2 data_type(size),
  column_name3 data_type(size),
  ....
);
```

Example

```
CREATE TABLE Persons
(
  PersonID int,
  LastName varchar(255),
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255)
);
```

✓ *Inserting Data into table*

```
Insert into <table_name> (col_name1 , col_name2)
Values (<val1> , <val2>)
```

Example

```
INSERT INTO Employee
(emp_id, emp_name, emp_address)
VALUES
(001, "John Doe", "1 River Walk, Green street")
```

We can copy all columns from one table to another, existing table:

```
INSERT INTO table2
SELECT * FROM table1;
```

Or we can copy only the columns we want to into another, existing table:

```
INSERT INTO table2
(column_name(s))
SELECT column_name(s)
FROM table1;
```

Example

```
INSERT INTO Customers (CustomerName, Country)
SELECT SupplierName, Country FROM Suppliers;
```

Example

```
INSERT INTO Customers (CustomerName, Country)
SELECT SupplierName, Country FROM Suppliers
WHERE Country='Germany';
```

✓ Modifying Structure of table

The structure of table can be modified by using the ALTER TABLE command.

It works by making a temporary copy of the original table. The alteration is performed on the copy, then the original table is deleted and new one is renamed.

While Alter Table is executing the original table is still readable by users (not writable).

- *Adding New column*

```
ALTER TABLE <table_name>
ADD (col_name <datatype> (<size>),
    (col_name <datatype> (<size>))
```

Example

```
ALTER TABLE Persons
ADD (DateOfBirth date)
```

- *Dropping a column*

```
ALTER TABLE <table_name>
DROP COLUMN <column_name>
```

Example

```
ALTER TABLE Persons
DROP COLUMN DateOfBirth
```

- *Modifying Existing column*

```
ALTER TABLE <table_name>
MODIFY (<column_name> <new_datatype>(<size>))
```

Example

```
ALTER TABLE Persons  
MODIFY COLUMN DateOfBirth year
```

- *Rename, Truncate & Drop table*

```
RENAME <table1> to <table2>  
TRUNCATE TABLE <table_name>  
DROP TABLE <table_name>
```

Q1- What is different between Delete Truncate and drop?

The DELETE command is used to remove rows from a table. A WHERE clause can be used to only remove some rows.

TRUNCATE removes all rows from a table. The operation cannot be rolled back

The DROP command removes a table from the database. All the tables' rows, indexes and privileges will also be removed. No DML triggers will be fired.

Q2- How to display user tables in schema and table structure?

```
SELECT * from TAB;
```

```
DESCRIBE <table_name>
```

🔗 INTERACTIVE SQL PART – 2

Data constraints :

The business rules which are enforced on data being stored in a table are called constraints. Even if a single column of the records being entered into a table fails a constraint, the entire record is rejected and not stored in the table.

Constraints can be specified when the table is created (inside the CREATE TABLE statement) or after the table is created (inside the ALTER TABLE statement).

```
CREATE TABLE table_name
(
column_name1 data_type(size) constraint_name,
column_name2 data_type(size) constraint_name,
column_name3 data_type(size) constraint_name,
....
);
```

⇒ Types of Data Constraints :

There are two types of data constraints

- 1) I/O constraints – it defines the speed at which data can be inserted or extracted from a table.
- 2) Business rule constraints – it defines the business rule which has to be followed.

✓ I/O constraints

- UNIQUE - Ensures that each row for a column must have a unique value
- PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Ensures that a column (or combination of two or more columns) has a unique identity which helps to find a particular record in a table more easily and quickly
- FOREIGN KEY - Ensures the referential integrity of the data in one table to match values in another table

✓ Business rule constraints

- NOT NULL - Indicates that a column cannot store a NULL value
- CHECK - Ensures that the value in a column meets a specific condition

```
<column_name> <data_type>(<size>) CHECK (<logical expression>)
```

Q-3 What are the different levels of data integrity in SQL Server?

* Entity Integrity

Entity Integrity ensures that there are no duplicate rows in a table.

* Domain Integrity

Domain Integrity enforces valid entries for a given column by restricting the type, the format, or the range of possible values.

* Referential integrity

Referential integrity ensures that rows cannot be deleted, which are used by other records.

Q-4 How can we add constraints to existing table?

Whenever we add constraint to existing table, Oracle checks that constraints against existing data of that column and if any of the existing data is violating the constraint that will not be applied.

ALTER table statement is used to add constraints to existing table.

MODIFY clause is used to add column level constraints.

ADD clause is used to add table level constraints.

Example

```
ALTER TABLE Persons  
ADD CONSTRAINT uc_PersonID UNIQUE (P_Id, LastName)
```

```
ALTER TABLE Persons  
DROP CONSTRAINT uc_PersonID
```

Q-5 Where the integrity constraints are stored in data dictionary?

The integrity constraints are stored in below table
USER_CONSTRAINTS

Q-6 How will you activate/deactivate integrity constraints?

```
ALTER TABLE <table_name>  
ENABLE/DISABLE CONSTRAINT
```

📖 INTERACTIVE SQL PART – 3

✓ Grouping table data

Group By clause tells oracle to group rows based on distinct values that exist for specified columns.

Syntax

```
SELECT <column_name>, aggregate_function(<column_name>)
FROM table_name
WHERE <condition>
GROUP BY <column_name>;
```

✓ The Having clause

The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions.

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE <condition>
GROUP BY column_name
HAVING aggregate_function(column_name) <condition>;
```

✓ The Oracle DUAL table

Dual is small oracle worktable , which consists of only one row and one column , and contains value x in that column.

Example

```
Select 2*2 from DUAL
Select sysdate from DUAL
```

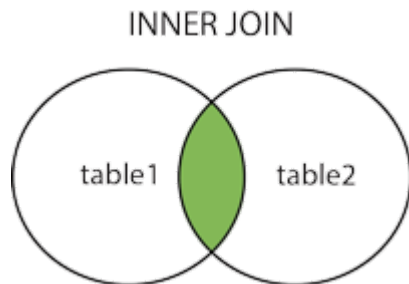
✓ Oracle Joins

An SQL JOIN clause is used to combine rows from two or more tables, based on a common field between them. If a join involves more than two tables then oracle joins first two tables based on the join condition then compare the result with next table and so on.

Types of Joins

⇒ Inner/Equi join

An SQL INNER JOIN return all rows from multiple tables where the join condition is met.



Syntax

```
SELECT column_name(s)
FROM table_name1
INNER JOIN table_name2
ON table_name1.column_name=table_name2.column_name
```

-- for example:

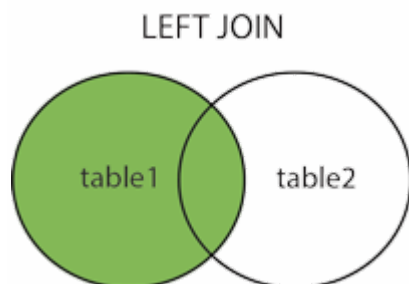
```
SELECT Person.LastName, Person.FirstName, Sales.OrderNo
FROM Person
INNER JOIN Sales
ON Person.P_Id = Sales.P_Id
```

⇒ Outer (LEFT,RIGHT,FULL) join

The LEFT JOIN keyword returns all rows from the left table (table1), with the matching rows in the right table (table2). The result is NULL in the right side when there is no match.

Syntax

```
SELECT column_name(s)
FROM table1
LEFT OUTER JOIN table2
ON table1.column_name=table2.column_name;
```

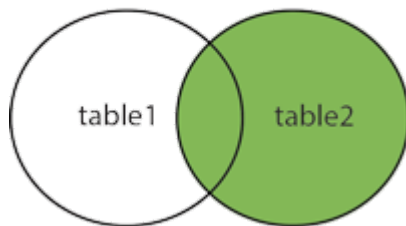


The **RIGHT JOIN** keyword returns all rows from the right table (table2), with the matching rows in the left table (table1). The result is NULL in the left side when there is no match.

Syntax

```
SELECT column_name(s)
FROM table1
RIGHT OUTER JOIN table2
ON table1.column_name=table2.column_name;
```

RIGHT JOIN



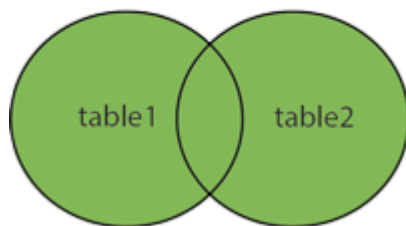
The **FULL OUTER JOIN** keyword returns all rows from the left table (table1) and from the right table (table2).

The FULL OUTER JOIN keyword combines the result of both LEFT and RIGHT joins.

Syntax

```
SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2
ON table1.column_name=table2.column_name;
```

FULL OUTER JOIN



⇒ Cross/non-equi join

A **CROSS JOIN** is a JOIN operation that produces the Cartesian product of two tables. Unlike other JOIN operators, it does not let you specify a join clause. You may, however, specify a WHERE clause in the SELECT statement.

-- for example:

```
SELECT * FROM CITIES CROSS JOIN FLIGHTS
WHERE CITIES.AIRPORT = FLIGHTS.ORIG_AIRPORT
```

Q-7 How to apply join in more than two table?

Suppose we have A,B & C table and a,b,c.d... columns.

```
SELECT *  
FROM A [INNER/LEFT/RIGHT/OUTER] JOIN  
      B ON [a/b].IDCols = [a/b].IDCols [INNER/LEFT/RIGHT/OUTER] JOIN  
      C ON [a/b/c].IDCols = [a/b/c].IDCols
```

Q-8 What is difference between Where and Having clause?

You cannot use Where clause to restrict the groups, Where clause is used to restrict rows and you cannot use GROUP functions in Where clause.

To restrict group we use Having clause.

-- for example:

```
Select deptno,AVG(sal) from emp  
Where AVG(sal) > 4000 ---Incorrect  
Having AVG(sal) > 4000 --- correct
```

□ INTERACTIVE SQL PART – 4

Advanced SQL – SQL performance tuning

⇒ *Index*

Indexes are special lookup tables that the database search engine can use to speed up data retrieval. Simply put, an index is a pointer to data in a table. An index in a database is very similar to an index in the back of a book.

When Index is created the oracle engine locates an associated record in the table using ROWID found in table.

✓ The CREATE INDEX Command:

The basic syntax of CREATE INDEX is as follows:

```
CREATE INDEX index_name ON table_name;
```

✓ Single-Column Indexes:

A single-column index is one that is created based on only one table column and it allows duplicate value. The basic syntax is as follows:

```
CREATE INDEX index_name  
ON table_name (column_name);
```

✓ Unique Indexes:

Unique indexes are used not only for performance, but also for data integrity. A unique index does not allow any duplicate values to be inserted into the table. The basic syntax is as follows:

```
CREATE UNIQUE INDEX index_name
```

```
on table_name (column_name);
```

✓ Composite Indexes:

A composite index is an index on two or more columns of a table. The basic syntax is as follows:

```
CREATE INDEX index_name  
on table_name (column1, column2);
```

✓ The DROP INDEX Command:

An index can be dropped using SQL DROP command. Care should be taken when dropping an index because performance may be slowed or improved.

The basic syntax is as follows:

```
DROP INDEX index_name;
```

Q-9 What is ROWID?

ROWID indicates exactly where the record is stored in table. Each table in Oracle database internally has a pseudocolumn named ROWID.

The value of the pseudocolumn ROWID cannot be set or deleted using INSERT or UPDATE.

The ROWID format used by Oracle for restricted format is as follows

BBBBBBB.RRRR.FFFF

Here FFFF is a unique number of data file.

BBBBBBB is block no in which record is stored.

RRRR is a unique record number.

Q-10 What is Rowid & Rownum?

Rowid is a system generated 18 character long unique identifier that is created for every record in the database.

Rownum is the pseudocolumn which returns row number of records. This is logical number determined at the time of query.

Q-11 What is NVL and NVL2?

NVL(comm,0) it will return comm if comm is not null otherwise it returns 0

NVL2(exp1,exp2,exp3) it will return exp2 if exp1 is not null otherwise it will return exp3.

Eg. NVL2(comm,sal+comm,sal)

Q-12 How index are created?

Automatically – A Unique Index is created automatically when we define primary or unique key on table.

Manually – User can create nonunique index.

Q-13 When to create index ?

- ⇒ When table is large
- ⇒ Column contains large no of null values.
- ⇒ One or more column is frequently used.
- ⇒ The table is not updated frequently.

Q-14 Can we modify index ?

We can't , to change an index we must drop and recreate it.

Q-15 How are Index updated ?

Changes to table data are automatically incorporated into all relevant indexes.

Q-16 Can we create index on views ?

No , we can't.

Q-17 What are the different types of index ?

Clustered Index: A clustered index is an index where the leaf level of the index contains the actual data rows of the table and It sorts and stores the data rows of the table or view in order based on the clustered index key. Index is defined for one or more columns called INDEX key. There can be only one clustered index per table and it can be combination of multiple columns.

When to use cluster indexes –

Queries that return a large percentage of the columns in the table

Queries that return a single row based on the clustered index key

Queries that return range-based data

Non-Clustered Index: A non-cluster index does not contain the entire data row at the leaf level. It contains just the columns defined in the index and a pointer or key to the actual data row. Single table supports up to 999 non-cluster indexes.

When to use non-cluster indexes –

Queries that return few rows

Queries that can be covered by the index

Covering Index: Covering indexes are those which improves the query performance and meets requirement of the query using non-cluster indexes. This is to encourage DBAs to create more non-cluster index rather than using cluster indexes and avoid pressure on cluster indexes.

Filtered Index: A filtered index is an optimized non-clustered index especially suited to cover queries that select from a well-defined subset of data. It uses a filter predicate to index a portion of rows in the table. A well-designed filtered index can improve query performance as well as reduce index maintenance and storage costs compared with full-table indexes. Filtered indexes are also useful in conjunction with another feature called sparse column. By creating a filtered index and specifying that the WHERE column IS NOT NULL, you create B-tree with only those rows that contain data

Advantage of using Filtered indexes

improved query performance and plan quality

Reduced index maintenance costs

Reduced index storage costs

When to use filter indexes –

When combined with sparse columns to locate specific non-null rows
When queries that are a small subset of the rows are selected often

Q-18 What is the difference between B-tree index and Bit map index ?

Bitmap index:

A type of index that uses a string of bits to quickly locate rows in a table.

Bitmap indexes are normally used to index low cardinality columns(*cardinality refers to the uniqueness of data values contained in a column. High cardinality means that the column contains a large percentage of totally unique values. Low cardinality means that the column contains a lot of “repeats” in its data range.*) in a warehouse environment.

Btree index:

A type of index that uses a balanced tree structure for efficient record retrieval.

B-tree indexes store key data in ascending or descending order.

Difference Between B-Tree Index & Bitmap Index:

1. B-tree Index has High cardinality values(usually when we have repeated columns), where as Bitmap Index has Low Cardinality values(usually when column have few values like true/false).
2. B-tree Index is useful for OLTP, where as Bitmap Index is useful for Dataware Housing.

Adv:

Bitmap indexes are more advantageous than b-tree indexes, when the table has millions of rows and the key columns have low cardinality.

⇒ Views

A view is a logical representation of another table or combination of tables. A view derives its data from the tables on which it is based. These tables are called base tables. Base tables might in turn be actual tables or might be views themselves. All operations performed on a view actually affect the base table of the view. You can use views in almost the same way as tables. You can query, update, insert into, and delete from views, just as you can standard tables.

```
CREATE VIEW <view_name> AS
SELECT <col1>, <col2>..
FROM <table_name>
WHERE <condition>
```

❓ Altering Views

You use the ALTER VIEW statement only to explicitly recompile a view that is invalid.

❓ Dropping Views

You can drop any view contained in your schema. For example, the following statement drops the emp_dept view:

```
DROP VIEW emp_dept;
```

❓ Replacing View

You cannot use an ALTER VIEW statement to change the definition of a view. You can replace views in the following ways:

- You can drop and re-create the view.

Caution:

When a view is dropped, all grants of corresponding object privileges are revoked from roles and users. After the view is re-created, privileges must be regranted.

Q-19 Can a view based on another view ?

YES

Q-20 What are the advantages of a view ?

- Provide an additional level of security.
- Simplify commands for the user.
- Store complex queries
- To hide complexity of calculations.

Q-21 What is updatable view ?

View on which data manipulation can be done are called updateable views.

A view is only updateable when

- View defined from single table
- All the primary key column and all the NOT NULL column must be included or excluded in the view.

⇒ Snapshot

A snapshot is recent copy of a table from database or in some cases , a subset of rows/column of a table.

Snapshot is the object created in local database having the data of remote database.While creating the snapshot you need to mention the frequency to refresh them.

Q-22 What is the advantage of snapshot ?

Using snapshot you can eliminate the needs to retrieve the data from remote database. This helps to reduce network load.

⇒ *Clusters*

Clusters are groups of one or more tables physically stored together to share common columns and are often used together.

Q-23 What is cluster key?

The related column of the table in a cluster are called the cluster key.

Q-24 What is index cluster?

A cluster with an index on the cluster key.

Q-25 Why clusters are created?

It helps to save disk space

It helps to retrieve the data using joins will be faster.

⇒ *Database Locks*

Q-26 What is locking, advantages of locking and types of locking in Oracle?

Locking is a mechanism to ensure data integrity while allowing maximum concurrent access to data. It is used to implement concurrency control when multiple users access table to manipulate its data at the same time.

--Advantage :

- a. Avoids deadlock conditions
- b. Avoids clashes in capturing the resources

--Types of locking

Shared Lock

This type is placed on a record when the record is being viewed (select statement).

Exclusive lock

This is placed when write operation is performed (Insert, Update & delete statement).

Q-27 What is Implicit and Explicit lock?

The Oracle engine automatically lock table data while executing SQL statements. This type of locking is called Implicit locking.

Oracle users can also explicitly lock the table data this type of locking is called explicit lock.

Q-28 What is different levels of lock in Oracle?

--Row Level

If the where clause evaluates to only one row in the table, a row level lock is used.

--Page Level

If the where clause evaluates to a set of data, a page level lock is used.

--Table level

If there is no where clause, a table level lock is used.

Q-29 What is deadlock?

A deadlock is a situation that can occur in a multiuser system that causes some number of transactions to be unable to continue work.

A deadlock can occur when two or more users are waiting for the data locked by each other.

Q-30 What is NOWAIT in Oracle?

In order to avoid unnecessary waiting time, a NOWAIT option can be used to inform the Oracle engine to terminate the SQL statement if the record has already been locked. If this happens, the Oracle engine terminates the running DML and comes up with a message indicating that the resource is busy.

Q-31 What are the methods to release locks?

Locks are released under the following circumstances:

--The transaction is committed successfully.

--A rollback is performed.

--savepoint will release lockset after the specified savepoint.

□ PL/SQL PART – 1

PL/SQL stands for Procedural Language extension of SQL.

PL/SQL is a combination of SQL along with the procedural features of programming languages. It was developed by Oracle Corporation in the early 90's to enhance the capabilities of SQL.

PL/SQL Block consists of three sections:

- The Declaration section (optional).
- The Execution section (mandatory).
- The Exception (or Error) Handling section (optional).

Example

```
DECLARE
    <Variable declaration>
BEGIN
    <Program Execution>
EXCEPTION
    <Exception handling>
END;
```

□ These are the Advantages of PL/SQL

- **Block Structures:** PL SQL consists of blocks of code, which can be nested within each other. Each block forms a unit of a task or a logical module. PL/SQL Blocks can be stored in the database and reused.
- **Procedural Language Capability:** PL SQL consists of procedural language constructs such as conditional statements (if else statements) and loops like (FOR loops).
- **Better Performance:** PL SQL engine processes multiple SQL statements simultaneously as a single block, thereby reducing network traffic.
- **Error Handling:** PL/SQL handles errors or exceptions effectively during the execution of a PL/SQL program. Once an exception is caught, specific actions can be taken depending upon the type of the exception or it can be displayed to the user with a message.

Q-32 What is PL/SQL variables?

A variable is nothing but a name given to a storage area that our programs can manipulate. Each variable in PL/SQL has a specific data type, which determines the size and layout of the variable's memory; the range of values that can be stored within that memory and the set of operations that can be applied to the variable.

Syntax

```
variable_name datatype [NOT NULL := value ];
```

-- for example:

```
DECLARE
    a integer := 10;
    b integer := 20;
    c integer;
    f real;
BEGIN
    c := a + b;
    dbms_output.put_line('Value of c: ' || c);
    f := 70.0/3.0;
    dbms_output.put_line('Value of f: ' || f);
END;
```

Q-33 What is local variable and global variable in PL/SQL?

- **Local variables** - variables declared in an inner block and not accessible to outer blocks.
- **Global variables** - variables declared in the outermost block or a package.

-- for example:

```
DECLARE
    -- Global variables
    num1 number := 95;
    num2 number := 85;
BEGIN
    dbms_output.put_line('Outer Variable num1: ' || num1);
    dbms_output.put_line('Outer Variable num2: ' || num2);
    DECLARE
        -- Local variables
        num1 number := 195;
        num2 number := 185;
    BEGIN
        dbms_output.put_line('Inner Variable num1: ' || num1);
        dbms_output.put_line('Inner Variable num2: ' || num2);
    END;
END;
```

Q-34 What is PL/SQL constants?

A constant holds a value that once declared, does not change in the program.

A constant is declared using the **CONSTANT** keyword. It requires an initial value and does not allow that value to be changed.

Syntax

```
constant_name CONSTANT datatype := VALUE;
```

-- for example:

```
PI CONSTANT NUMBER := 3.141592654;
```

Q-35 What is PL/SQL loop?

There may be a situation when you need to execute a block of code several number of times. In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on.

-- for example:

```
DECLARE
    i number(1);
    j number(1);
BEGIN
    << outer_loop >>
    FOR i IN 1..3 LOOP
        << inner_loop >>
        FOR j IN 1..3 LOOP
            dbms_output.put_line('i is: ' || i || ' and j is: ' || j);
        END loop inner_loop;
    END loop outer_loop;
END;
```

⇒ Cursors

Oracle creates a memory area, known as context area, for processing an SQL statement, which contains all information needed for processing the statement, for example, number of rows processed, etc.

A cursor is a pointer to this context area. PL/SQL controls the context area through a cursor. A cursor holds the rows (one or more) returned by a SQL statement. The set of rows the cursor holds is referred to as the active set.

You can name a cursor so that it could be referred to in a program to fetch and process the rows returned by the SQL statement, one at a time. There are two types of cursors:

- Implicit cursors
- Explicit cursors

Q-36 What is Implicit cursor & its attributes?

Implicit cursors are automatically created by Oracle whenever an SQL statement is executed, when there is no explicit cursor for the statement. Programmers cannot control the implicit cursors and the information in it.

Whenever a DML statement (INSERT, UPDATE and DELETE) is issued, an implicit cursor is associated with this statement.

Oracle provides few attributes called as implicit cursor attributes to check the status of DML operations. The cursor attributes available are %FOUND, %NOTFOUND, %ROWCOUNT, and %ISOPEN.

Attribute	Description
%FOUND	Returns TRUE if an INSERT, UPDATE, or DELETE statement affected one or more rows or a SELECT INTO statement returned one or more rows. Otherwise, it returns FALSE.
%NOTFOUND	The logical opposite of %FOUND. It returns TRUE if an INSERT, UPDATE, or DELETE statement affected no rows, or a SELECT INTO statement returned no rows. Otherwise, it returns FALSE.
%ISOPEN	Always returns FALSE for implicit cursors, because Oracle closes the SQL cursor automatically after executing its associated SQL statement.
%ROWCOUNT	Returns the number of rows affected by an INSERT, UPDATE, or DELETE statement, or returned by a SELECT INTO statement.

Any SQL cursor attribute will be accessed as sql%attribute_name as shown below in the example.

Example:

We will be using the CUSTOMERS table we had created and used in the previous chapters.

```
Select * from customers;
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00

The following program would update the table and increase salary of each customer by 500 and use the SQL%ROWCOUNT attribute to determine the number of rows affected:

```
DECLARE
    total_rows number(2);
BEGIN
    UPDATE customers
    SET salary = salary + 500;
    IF sql%notfound THEN
        dbms_output.put_line('no customers selected');
    ELSIF sql%found THEN
        total_rows := sql%rowcount;
        dbms_output.put_line( total_rows || ' customers selected ');
    END IF;
END;
/
```

When the above code is executed at SQL prompt, it produces the following result:

```
6 customers selected
```

```
PL/SQL procedure successfully completed.
```

If you check the records in customers table, you will find that the rows have been updated:

```
Select * from customers;
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2500.00
2	Khilan	25	Delhi	2000.00
3	kaushik	23	Kota	2500.00
4	Chaitali	25	Mumbai	7000.00
5	Hardik	27	Bhopal	9000.00
6	Komal	22	MP	5000.00

Q-37 Explain explicit cursor?

Explicit cursors are programmer defined cursors for gaining more control over the context area. An explicit cursor should be defined in the declaration section of the PL/SQL Block. It is created on a SELECT Statement which returns more than one row.

The syntax for creating an explicit cursor is :

```
CURSOR cursor_name IS select_statement;
```

Working with an explicit cursor involves four steps:

- Declaring the cursor for initializing in the memory
- Opening the cursor for allocating memory
- Fetching the cursor for retrieving data
- Closing the cursor to release allocated memory

Declaring the Cursor

Declaring the cursor defines the cursor with a name and the associated SELECT statement. For example:

```
CURSOR c_customers IS  
SELECT id, name, address FROM customers;
```

Opening the Cursor

Opening the cursor allocates memory for the cursor and makes it ready for fetching the rows returned by the SQL statement into it. For example, we will open above-defined cursor as follows:

```
OPEN c_customers;
```

Fetching the Cursor

Fetching the cursor involves accessing one row at a time. For example we will fetch rows from the above-opened cursor as follows:

```
FETCH c_customers INTO c_id, c_name, c_addr;
```

Closing the Cursor

Closing the cursor means releasing the allocated memory. For example, we will close above-opened cursor as follows:

```
CLOSE c_customers;
```

Example:

Following is a complete example to illustrate the concepts of explicit cursors:

```
DECLARE
  c_id customers.id%type;
  c_name customers.name%type;
  c_addr customers.address%type;
  CURSOR c_customers IS
    SELECT id, name, address FROM customers;
BEGIN
  OPEN c_customers;
  LOOP
    FETCH c_customers INTO c_id, c_name, c_addr;
    EXIT WHEN c_customers%notfound;
    dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);
  END LOOP;
  CLOSE c_customers;
END;
```

When the above code is executed at SQL prompt, it produces the following result:

```
1 Ramesh Ahmedabad
2 Khilan Delhi
3 kaushik Kota
4 Chaitali Mumbai
5 Hardik Bhopal
6 Komal MP
```

PL/SQL procedure successfully completed.

Q-38 Can you pass a parameter to a cursor?

Explicit cursors can take parameters, as the example below shows. A cursor parameter can appear in a query wherever a constant can appear.

```
CURSOR c1 (median IN NUMBER) IS
```

```
SELECT job, ename FROM emp WHERE sal > median;
```

Q-39 Can you use %RowCount as a parameter to a cursor?

Yes

Q-40 If all the values from a cursor have been fetched and another fetch is issued, the output will be : error, last record or first record ?

Last Record

Q-41 Show code of a cursor for loop.

Cursor declares %ROWTYPE as loop index implicitly. It then opens a cursor, gets rows of values from the active set in fields of the record and shuts when all records are processed.

With cursor FOR loops , no opening , fetching , closing of cursor is required.

Eg.

```
FOR smp_rec IN C1 LOOP
totalsal=totalsal+smp_recsal;
ENDLOOP;
```

Q-42. If a cursor is open, how can we find in a PL/SQL Block?

The %ISOPEN cursor status variable can be used.

Q-43 Explain parameterized cursor ?

Simple cursor is used to fetch pre-determined set of records. Commercial applications require that the query , which defines the cursor be generic and the data that is retrived from the table be allowed to change according to the need

Oracle recognized this and permits the creation of parameterized cursor for use. The contents of a parameterized cursor will constantly change depending upon the values passed to its parameter.

⇒ Records

A PL/SQL record is a data structure that can hold data items of different kinds. Records consist of different fields, similar to a row of a database table.

PL/SQL can handle the following types of records:

- Table-based
- Cursor-based records
- User-defined records

The General Syntax to define a composite datatype is:

```
TYPE record_type_name IS RECORD
(first_col_name column_datatype,
second_col_name column_datatype, ...);
```

Table-Based Records

The %ROWTYPE attribute enables a programmer to create table-based and cursor-based records.

The following example would illustrate the concept of table-based records. We will be using the CUSTOMERS table we had created and used in the previous chapters:

```
DECLARE
    customer_rec customers%rowtype;
BEGIN
    SELECT * INTO customer_rec
    FROM customers
    WHERE id = 5;

    dbms_output.put_line('Customer ID: ' || customer_rec.id);
    dbms_output.put_line('Customer Name: ' || customer_rec.name);
    dbms_output.put_line('Customer Address: ' || customer_rec.address);
    dbms_output.put_line('Customer Salary: ' || customer_rec.salary);
END;
/
```

When the above code is executed at SQL prompt, it produces the following result:

```
Customer ID: 5
Customer Name: Hardik
Customer Address: Bhopal
Customer Salary: 9000

PL/SQL procedure successfully completed.
```

Cursor-Based Records

The following example would illustrate the concept of cursor-based records. We will be using the CUSTOMERS table we had created and used in the previous chapters:

```
DECLARE
    CURSOR customer_cur IS
        SELECT id, name, address
        FROM customers;
    customer_rec customer_cur%rowtype;
BEGIN
    OPEN customer_cur;
    LOOP
        FETCH customer_cur INTO customer_rec;
        EXIT WHEN customer_cur%notfound;
        DBMS_OUTPUT.put_line(customer_rec.id || ' ' || customer_rec.name);
    END LOOP;
END;
/
```

When the above code is executed at SQL prompt, it produces the following result:

```
1 Ramesh
2 Khilan
3 kaushik
4 Chaitali
5 Hardik
6 Komal

PL/SQL procedure successfully completed.
```

User-Defined Records

PL/SQL provides a user-defined record type that allows you to define different record structures. Records consist of different fields. Suppose you want to keep track of your books in a library. You might want to track the following attributes about each book:

- Title
- Author
- Subject
- Book ID

The record type is defined as:

```
TYPE
type_name IS RECORD
( field_name1 datatype1 [NOT NULL] [:= DEFAULT EXPRESSION],
  field_name2 datatype2 [NOT NULL] [:= DEFAULT EXPRESSION],
  ...
  field_nameN datatypeN [NOT NULL] [:= DEFAULT EXPRESSION]);
record-name type_name;
```

Here is the way you would declare the Book record:

```
DECLARE
TYPE books IS RECORD
(title varchar(50),
 author varchar(50),
 subject varchar(100),
 book_id number);
book1 books;
book2 books;
```

⇒ Procedure

A procedure or function is a logical set of SQL and PL/SQL statements that performs a specific task. Procedure and function are stored in the Oracle database.

Creating a Procedure

A procedure is created with the CREATE OR REPLACE PROCEDURE statement. The simplified syntax for the CREATE OR REPLACE PROCEDURE statement is as follows:

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[(parameter_name [IN | OUT | IN OUT] type [, ...])]
{IS | AS}
BEGIN
    < procedure_body >
END procedure_name;
```

Where,

- procedure-name specifies the name of the procedure.
- [OR REPLACE] option allows modifying an existing procedure.
- The optional parameter list contains name, mode and types of the parameters. IN represents that value will be passed from outside and OUT represents that this parameter will be used to return a value outside of the procedure.

- procedure-body contains the executable part.
- The AS keyword is used instead of the IS keyword for creating a standalone procedure.

Example:

The following example creates a simple procedure that displays the string 'Hello World!' on the screen when executed.

```
CREATE OR REPLACE PROCEDURE greetings
AS
BEGIN
    dbms_output.put_line('Hello World!');
END;
/
```

Executing a Standalone Procedure

A standalone procedure can be called in two ways:

- Using the EXECUTE keyword
- Calling the name of the procedure from a PL/SQL block.

The above procedure named 'greetings' can be called with the EXECUTE keyword as:

```
EXECUTE greetings;
```

The procedure can also be called from another PL/SQL block:

```
BEGIN
    greetings;
END;
/
```

Deleting a Standalone Procedure

A standalone procedure is deleted with the DROP PROCEDURE statement. Syntax for deleting a procedure is:

```
DROP PROCEDURE procedure-name;
```

So you can drop greetings procedure by using the following statement:

```
BEGIN
    DROP PROCEDURE greetings;
END;
/
```

Q-44 What is parameter mode in PL/SQL?

We can pass parameters to procedures in three ways.

1) IN-parameters

An IN parameter lets you pass a value to the subprogram. It is a read-only parameter. Inside the subprogram, an IN parameter acts like a constant. It cannot be assigned a value.

2) OUT-parameters

An OUT parameter returns a value to the calling program. Inside the subprogram, an OUT parameter acts like a variable. You can change its value and reference the value after assigning it.

3) IN OUT-parameters

An IN OUT parameter passes an initial value to a subprogram and returns an updated value to the caller. It can be assigned a value and its value can be read.

IN & OUT Mode Example 1

This program finds the minimum of two values, here procedure takes two numbers using IN mode and returns their minimum using OUT parameters.

```
DECLARE
    a number;
    b number;
    c number;

PROCEDURE findMin(x IN number, y IN number, z OUT number) IS
BEGIN
    IF x < y THEN
        z := x;
    ELSE
        z := y;
    END IF;
END;

BEGIN
    a := 23;
    b := 45;
    findMin(a, b, c);
    dbms_output.put_line(' Minimum of (23, 45) : ' || c);
END;
/
```

When the above code is executed at SQL prompt, it produces the following result:

```
Minimum of (23, 45) : 23
```

PL/SQL procedure successfully completed.

IN & OUT Mode Example 2

This procedure computes the square of value of a passed value. This example shows how we can use same parameter to accept a value and then return another result.

```
DECLARE
    a number;

PROCEDURE squareNum(x IN OUT number) IS
BEGIN
    x := x * x;
END;
```

```

END;
BEGIN
    a:= 23;
    squareNum(a);
    dbms_output.put_line(' Square of (23): ' || a);
END;
/

```

When the above code is executed at SQL prompt, it produces the following result:

```
Square of (23): 529
```

```
PL/SQL procedure successfully completed.
```

⇒ *Function*

A PL/SQL function is same as a procedure except that it returns a value. Therefore, all the discussions of the previous chapter are true for functions too.

Creating a Function

A standalone function is created using the CREATE FUNCTION statement. The simplified syntax for the CREATE OR REPLACE PROCEDURE statement is as follows:

```

CREATE [OR REPLACE] FUNCTION function_name
[(parameter_name [IN | OUT | IN OUT] type [, ...])]
RETURN return_datatype
{IS | AS}
BEGIN
    < function_body >
END [function_name];

```

Example 1:

The following example illustrates creating and calling a standalone function. This function returns the total number of CUSTOMERS in the customers table.

```
Select * from customers;
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00

```

CREATE OR REPLACE FUNCTION totalCustomers
RETURN number IS

```

```

    total number(2) := 0;
BEGIN
    SELECT count(*) into total
    FROM customers;

    RETURN total;
END;
/

```

When above code is executed using SQL prompt, it will produce the following result:

```
Function created.
```

Calling a Function

Use a function, you will have to call that function to perform the defined task. When a program calls a function, program control is transferred to the called function.

A called function performs defined task and when its return statement is executed or when its last end statement is reached, it returns program control back to the main program.

To call a function you simply need to pass the required parameters along with function name and if function returns a value then you can store returned value. Following program calls the function totalCustomers from an anonymous block:

```

DECLARE
    c number(2);
BEGIN
    c := totalCustomers();
    dbms_output.put_line('Total no. of Customers: ' || c);
END;
/

```

When the above code is executed at SQL prompt, it produces the following result:

```
Total no. of Customers: 6
```

```
PL/SQL procedure successfully completed.
```

Example 2:

The following is one more example which demonstrates Declaring, Defining, and Invoking a Simple PL/SQL Function that computes and returns the maximum of two values.

```

DECLARE
    a number;
    b number;
    c number;
FUNCTION findMax(x IN number, y IN number)
RETURN number
IS
    z number;
BEGIN
    IF x > y THEN
        z := x;
    ELSE
        z := y;
    END IF;
END;

```

```

        END IF;

        RETURN z;
    END;
BEGIN
    a:= 23;
    b:= 45;

    c := findMax(a, b);
    dbms_output.put_line(' Maximum of (23,45): ' || c);
END;
/

```

When the above code is executed at SQL prompt, it produces the following result:

```
Maximum of (23,45): 45
```

```
PL/SQL procedure successfully completed.
```

Q-44 How does oracle engine creates a stored procedure function ?

When function or procedure is stored in oracle database a compilation process is performed. The compilation process does not display the errors. These errors can be viewed using the SELECT statement.

Select * from user_errors

The memory area where the oracle engine stores the compiled function or procedure is called System Global Area (SGA).

Q-45 What is the advantages of using procedure and function ?

- ⇒ Security
- ⇒ Performance
- ⇒ Memory Allocation
- ⇒ Productivity
- ⇒ Integrity

Q-46 What is difference between procedure and function ?

---Procedure

- ⇒ Do not contain RETURN clause in the header.
- ⇒ Can return none, one or many values with the help of OUT & IN OUT parameter.

---Function

- ⇒ Must contain a RETURN clause in the header.
- ⇒ Must return a value using the RETURN clause. Can also return one or more many values with the help of OUT and IN OUT parameters.

Q-47 What is function and procedure overloading?

Overloading means creating multiple procedures or functions of the same name in a package, which take different numbers of arguments and / or where the arguments have different datatypes.

⇒ Exception

An error condition during a program execution is called an exception in PL/SQL. PL/SQL supports programmers to catch such conditions using EXCEPTION block in the program and an appropriate action is taken against the error condition. There are two types of exceptions:

- ⇒ System-defined exceptions
- ⇒ User-defined exceptions

Syntax for Exception Handling

The General Syntax for exception handling is as follows. The default exception will be handled using WHEN others THEN:

```
DECLARE
    <declarations section>
BEGIN
    <executable command(s)>
EXCEPTION
    <exception handling goes here >
    WHEN exception1 THEN
        exception1-handling-statements
    WHEN exception2 THEN
        exception2-handling-statements
    WHEN exception3 THEN
        exception3-handling-statements
    .....
    WHEN others THEN
        exception3-handling-statements
END;
```

Example

We will be using the CUSTOMERS table we had created and used in the previous chapters:

```
DECLARE
    c_id customers.id%type := 8;
    c_name customers.name%type;
    c_addr customers.address%type;
BEGIN
    SELECT name, address INTO c_name, c_addr
    FROM customers
    WHERE id = c_id;

    DBMS_OUTPUT.PUT_LINE ('Name: ' || c_name);
    DBMS_OUTPUT.PUT_LINE ('Address: ' || c_addr);
EXCEPTION
    WHEN no_data_found THEN
        dbms_output.put_line('No such customer!');
    WHEN others THEN
        dbms_output.put_line('Error!');
END;
/
```

Raising Exceptions

Exceptions are raised by the database server automatically whenever there is any internal database error, but exceptions can be raised explicitly by the programmer by using the command RAISE. Following is the simple syntax of raising an exception:

```
DECLARE
    exception_name EXCEPTION;
BEGIN
    IF condition THEN
        RAISE exception_name;
    END IF;
EXCEPTION
    WHEN exception_name THEN
        statement;
END;
```

User-defined Exceptions

PL/SQL allows you to define your own exceptions according to the need of your program. A user-defined exception must be declared and then raised explicitly, using either a RAISE statement or the procedure DBMS_STANDARD.RAISE_APPLICATION_ERROR.

The syntax for declaring an exception is:

```
DECLARE
    my-exception EXCEPTION;
```

Example:

The following example illustrates the concept. This program asks for a customer ID, when the user enters an invalid ID, the exception invalid_id is raised.

```
DECLARE
    c_id customers.id%type := &cc_id;
    c_name customers.name%type;
    c_addr customers.address%type;

    -- user defined exception
    ex_invalid_id EXCEPTION;
BEGIN
    IF c_id <= 0 THEN
        RAISE ex_invalid_id;
    ELSE
        SELECT name, address INTO c_name, c_addr
        FROM customers
        WHERE id = c_id;

        DBMS_OUTPUT.PUT_LINE ('Name: ' || c_name);
        DBMS_OUTPUT.PUT_LINE ('Address: ' || c_addr);
    END IF;
EXCEPTION
    WHEN ex_invalid_id THEN
        dbms_output.put_line('ID must be greater than zero!');
    WHEN no_data_found THEN
        dbms_output.put_line('No such customer!');
    WHEN others THEN
        dbms_output.put_line('Error!');
END;
/
```

Q-48 What is named (predefined) oracle exceptions?

Set of oracle standard exceptions that are already named & do not need to declare.

Eg. NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, VALUE_ERROR, NO_DATA_FOUND

Q-49 What is the difference between SQLCODE & SQLERRM?

There are two built in functions which can be used to find out which exception occurred during an execution of a program unit.

SQLCODE : Returns the error number of the corresponding oracle error that was raised. Returns 1 for user defined exception.

SQLERRM : Returns the error message associated with the given error number, returns "USER DEFINED EXCEPTION" for user defined exceptions.

Q-50 How many exceptions are active at a time?

Only one

Q-51 What is OTHERS handler?

PL/SQL defines a special exception handler, known as OTHERS. This handler will execute for all raised exceptions that are not handled by any other handler.

⇒ Trigger

Triggers are stored programs, which are automatically executed or fired when some events occur. Triggers are, in fact, written to be executed in response to any of the following events:

- A database manipulation (DML) statement (DELETE, INSERT, or UPDATE).
- A database definition (DDL) statement (CREATE, ALTER, or DROP).
- A database operation (SERVERERROR, LOGON, LOGOFF, STARTUP, or SHUTDOWN).

Triggers could be defined on the table, view, schema, or database with which the event is associated.

Benefits of Triggers

Triggers can be written for the following purposes:

- Generating some derived column values automatically
- Enforcing referential integrity
- Event logging and storing information on table access
- Auditing
- Synchronous replication of tables
- Imposing security authorizations
- Preventing invalid transactions

Creating Triggers

```
CREATE [OR REPLACE ] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF }
{INSERT [OR] | UPDATE [OR] | DELETE}
[OF col_name]
ON table_name
[REFERENCING OLD AS o NEW AS n]
[FOR EACH ROW]
WHEN (condition)
DECLARE
    Declaration-statements
BEGIN
    Executable-statements
EXCEPTION
    Exception-handling-statements
END;
```

Example:

To start with, we will be using the CUSTOMERS table

```
Select * from customers;
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00

The following program creates a row level trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table. This trigger will display the salary difference between the old values and new values:

```
CREATE OR REPLACE TRIGGER display_salary_changes
BEFORE DELETE OR INSERT OR UPDATE ON customers
FOR EACH ROW
WHEN (NEW.ID > 0)
DECLARE
    sal_diff number;
BEGIN
    sal_diff := :NEW.salary - :OLD.salary;
    dbms_output.put_line('Old salary: ' || :OLD.salary);
    dbms_output.put_line('New salary: ' || :NEW.salary);
    dbms_output.put_line('Salary difference: ' || sal_diff);
END;
/
```

Here following two points are important and should be noted carefully:

- OLD and NEW references are not available for table level triggers, rather you can use them for record level triggers.

- If you want to query the table in the same trigger, then you should use the AFTER keyword, because triggers can query the table or change it again only after the initial changes are applied and the table is back in a consistent state.
- Above trigger has been written in such a way that it will fire before any DELETE or INSERT or UPDATE operation on the table, but you can write your trigger on a single or multiple operations, for example BEFORE DELETE, which will fire whenever a record will be deleted using DELETE operation on the table.

Triggering a Trigger

Let us perform some DML operations on the CUSTOMERS table. Here is one INSERT statement, which will create a new record in the table:

```
INSERT INTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)
VALUES (7, 'Kriti', 22, 'HP', 7500.00 );
```

When a record is created in CUSTOMERS table, above create trigger display_salary_changes will be fired and it will display the following result:

```
Old salary:
New salary: 7500
Salary difference:
```

Because this is a new record so old salary is not available and above result is coming as null. Now, let us perform one more DML operation on the CUSTOMERS table. Here is one UPDATE statement, which will update an existing record in the table:

```
UPDATE customers
SET salary = salary + 500
WHERE id = 2;
```

When a record is updated in CUSTOMERS table, above create trigger display_salary_changes will be fired and it will display the following result:

```
Old salary: 1500
New salary: 2000
Salary difference: 500
```

Types of PL/SQL Triggers

There are two types of triggers based on the which level it is triggered.

- 1) **Row level trigger** - An event is triggered for each row updated, inserted or deleted.
- 2) **Statement level trigger** - An event is triggered for each sql statement executed.

PL/SQL Trigger Execution Hierarchy

The following hierarchy is followed when a trigger is fired.

- 1) BEFORE statement trigger fires first.

- 2) Next BEFORE row level trigger fires, once for each row affected.
- 3) Then AFTER row level trigger fires once for each affected row. This events will alternates between BEFORE and AFTER row level triggers.
- 4) Finally the AFTER statement level trigger fires.

For Example: Let's create a table 'product_check' which we can use to store messages when triggers are fired.

```
CREATE TABLE product
(Message varchar2(50),
 Current_Date number(32)
);
```

Let's create a BEFORE and AFTER statement and row level triggers for the product table.

1) BEFORE UPDATE, Statement Level: This trigger will insert a record into the table 'product_check' before a sql update statement is executed, at the statement level.

```
CREATE or REPLACE TRIGGER Before_Update_Stat_product
BEFORE
UPDATE ON product
Begin
INSERT INTO product_check
Values('Before update, statement level',sysdate);
END;
/
```

2) BEFORE UPDATE, Row Level: This trigger will insert a record into the table 'product_check' before each row is updated.

```
CREATE or REPLACE TRIGGER Before_Update_Row_product
BEFORE
UPDATE ON product
FOR EACH ROW
BEGIN
INSERT INTO product_check
Values('Before update row level',sysdate);
END;
/
```

3) AFTER UPDATE, Statement Level: This trigger will insert a record into the table 'product_check' after a sql update statement is executed, at the statement level.

```
CREATE or REPLACE TRIGGER After_Update_Stat_product
AFTER
UPDATE ON product
BEGIN
INSERT INTO product_check
Values('After update, statement level', sysdate);
End;
/
```

4) AFTER UPDATE, Row Level: This trigger will insert a record into the table 'product_check' after each row is updated.

```

CREATE or REPLACE TRIGGER After_Update_Row_product
AFTER
insert On product
FOR EACH ROW
BEGIN
INSERT INTO product_check
Values('After update, Row level',sysdate);
END;
/

```

Now lets execute a update statement on table product.

```

UPDATE PRODUCT SET unit_price = 800
WHERE product_id in (100,101);

```

Lets check the data in 'product_check' table to see the order in which the trigger is fired.

```

SELECT * FROM product_check;

```

Output:

Message	Current_Date

Before update, statement level	26-Nov-2008
Before update, row level	26-Nov-2008
After update, Row level	26-Nov-2008
Before update, row level	26-Nov-2008
After update, Row level	26-Nov-2008
After update, statement level	26-Nov-2008

The above result shows 'before update' and 'after update' row level events have occurred twice, since two records were updated. But 'before update' and 'after update' statement level events are fired only once per sql statement.

The above rules apply similarly for INSERT and DELETE statements.

Q-52 What is difference between stored procedure and triggers?

Stored Procedure:

- ⇒ A procedure called explicitly by a user, application or a trigger.
- ⇒ Can not be inactive
- ⇒ Can have parameters
- ⇒ No restriction on the size of procedure.

Triggers

- ⇒ A trigger executes implicitly in response to an event.
- ⇒ We can enable and disable according to the need.
- ⇒ Can't have parameters.
- ⇒ There may be a restriction on the size of a trigger depending on the version of oracle used.

Q-53 What is trigger cascading?

When statement given in body of one trigger causes another trigger to fire is called trigger cascading , oracle allows max 32 triggers to cascade at a time.

Q-54 Which trigger is more efficient AFTER or BEFORE?

After row trigger is slightly more efficient than BEFORE row trigger because in BEFORE row trigger data blocks need to read once for the trigger and then again for the triggering statement.

Q-55 Can we commit or rollback in trigger?

No

Q-56 Can a trigger written for a view?

No

Q-57 What is mutating table error?

A mutating table is a table which currently being modified by an update,delete or insert statement.

This error occurs when we create a row level trigger on a table that attempts to access the same table inside the trigger body.

⇒ Package

PL/SQL packages are schema objects that groups logically related PL/SQL types, variables and subprograms.

A package will have two mandatory parts:

- Package specification

-- for example:

```
CREATE PACKAGE cust_sal AS
  PROCEDURE find_sal(c_id customers.id%TYPE);
END cust_sal;
/
```

- Package body or definition

-- for example:

```
CREATE OR REPLACE PACKAGE BODY cust_sal AS
  PROCEDURE find_sal(c_id customers.id%TYPE) IS
    c_sal customers.salary%TYPE;
  BEGIN
    SELECT salary INTO c_sal
    FROM customers
    WHERE id = c_id;
    dbms_output.put_line('Salary: ' || c_sal);
  END find_sal;
END cust_sal;
```

Advantages of Package

- ⇒ Modularity
- ⇒ Easier Application Design
- ⇒ Information hiding
- ⇒ Better Performance
- ⇒ Overloading

❓ DATABASE STRUCTURE

SQL Statements

1) Data retrieval

- ⇒ Select

2) DML (data manipulation language) ---*commit required*

- ⇒ Insert
- ⇒ Update
- ⇒ Delete
- ⇒ Merge

3) DDL (data definition language) ---*Automatic commit*

- ⇒ Create
- ⇒ Alter
- ⇒ Drop
- ⇒ Rename
- ⇒ Truncate

4) TCL (Transaction control language)

- ⇒ Commit
- ⇒ Rollback
- ⇒ Savepoint

5) DCL (Data control language) ---Automatic commit

- ⇒ Grant
- ⇒ Revoke

Q-58 What is an oracle database?

A database is a collection of data. An Oracle database holds its data in a coordinated set of files stored on disk, including following types:

- **Parameter Files** - The Oracle Parameter File (PFILE or SPFILE) holds initialization parameters which indicate where the control files are, how memory is allocated, how logging is performed, and other characteristics of the Oracle instance.
- **Control Files** - The Control File stores information needed to verify that the database is consistent, including a list of all database files making up the instance and other important data.
- **Redo Log Files** - The Redo Log files store a sequence of entries describing all actions taken against the database. This data is used to recover a database in the event of instance failure.
- **Data Files** - The Data Files contain blocks of data which store database objects (e.g., tables, indexes, materialized views, etc.) in the database.
- **Temp Files** - The Temp files contains data used temporarily including intermediate results, sort results and so forth.

Q-59 What is an Instance?

An instance is a collection of Oracle background processes and shared memory structures.

Memory Areas

- SGA - System Global Area
The SGA consists of the following four (five if MTS) parts:
 - Fixed Portion

- Variable Portion
 - Shared pool
 - java pool
- PGA - Process Global Area
 - UGA - User Global Area

Processes

Oracle uses many small (focused) processes to manage and control the Oracle instance. This allows for optimum execution on multi-processor systems using multi-core and multi-threaded technology. Some of these processes include:

- PMON - Process Monitor
- SMON - System Monitor
- ARCn - Redo Log Archiver
- LGWR - Redo Log Writer
- DBWn - Database Writer
- CKPT - Checkpoint process

Q-60 What is table space?

Disk space needs to be allocated for certain database objects (like tables and indexes). In Oracle, disk space from the operating system is allocated to tablespaces. Database objects are then created within a tablespace.

To list all tablespaces:

```
SELECT tablespace_name FROM dba_tablespaces;
```

Commands used to create new tablespaces:

```
CREATE TABLESPACE ts1 DATAFILE 'u01/oradata/orcl_ts1_01.dbf' SIZE 100M;
CREATE UNDO TABLESPACE undots1 DATAFILE 'u01/oradata/orcl_undots1_01.dbf' SIZE
100M;
CREATE TEMPORARY TABLESPACE temp1 TEMPFILE 'u01/oradata/orcl_temp1_01.dbf' SIZE
100M;
```

Q-61 What is schema objects?

Schema objects are created within a schema (Oracle user). Here are some of the object types that can be created:

- Table (heap, IOT, temporary, etc.)
- Index
- View
- Materialized View (snapshot)
- Sequence
- Synonym

- Cluster
- Trigger
- Procedure
- Function
- Package (containing procedures and functions)

Etc.

Q-62 What are the components of Physical database structure of Oracle Database?

ORACLE database is comprised of three types of files. One or more Data files, two or more Redo Log files, and one or more Control files.

Q-63 What are the components of Logical database structure of ORACLE database?

Tablespaces and the Database's Schema Objects.

Q-64 What is a Tablespace?

A database is divided into Logical Storage Unit called tablespaces. A tablespace is used to group related logical structures together.

Q-65 What is SYSTEM tablespace and When is it Created?

Every ORACLE database contains a tablespace named SYSTEM, which is automatically created when the database is created. The SYSTEM tablespace always contains the data dictionary tables for the entire database.

Q-66 Explain the relationship among Database, Tablespace and Data file.

Each database is logically divided into one or more tablespaces. One or more data files are explicitly created for each tablespace.

Q-67 what is Table?

A table is the basic unit of data storage in an ORACLE database. The tables of a database hold all of the user accessible data. Table data is stored in rows and columns.

Q-68 What is oracle synonyms?

A **synonym** is an alternative name (alias) for objects such as tables, views, sequences, stored procedures, and other database objects.

There are two types of Synonyms: Private and Public.

SYNTAX

```
CREATE [OR REPLACE] [PUBLIC] SYNONYM [schema .] synonym_name
```

```
FOR [schema .] object_name [@ dblink];
```

-- for example:

```
CREATE OR REPLACE PUBLIC SYNONYM suppliers  
FOR app.suppliers;
```

Q-69 What are synonyms used for?

- ⇒ Synonyms are used to mask the real name and owner of an object.
- ⇒ Provide public access to an object
- ⇒ Provide location transparency for tables, views or program units of a remote database.
- ⇒ Simplify the SQL statements for database users.

Q-70 What is Database Link?

A database link is a named object that describes a "path" from one database to another.
Private Database Link, Public Database Link & Network Database Link.

Q-71 What is Data Block?

ORACLE database's data is stored in data blocks. One data block corresponds to a specific number of bytes of physical database space on disk.

Q-72 How to define Data Block size?

A data block size is specified for each ORACLE database when the database is created. A database users and allocated free database space in ORACLE datablocks. Block size is specified in INIT.ORA file and can't be changed latter.

Q-73 What does a Control file Contain?

A Control file records the physical structure of the database. It contains the following information.

- Database Name
- Names and locations of a database's files and redolog files.
- Time stamp of database creation.

Q-74 What are the steps to process SQL statements?

All SQL statements have to go through various stages, some may be skipped.

Parse :

Every SQL statement must be parsed. Parsing the statement includes checking the statement's syntax and validating the statement.

Bind :

After parsing the oracle server knows the meaning of the oracle statement but still may not have enough information to execute the statement. The oracle server may need values for any bind variable in the statement. The process of obtaining these values is called binding variables.

Execute :

All this point, the oracle server has all necessary information and resources and the statement is executed.

Fetch :

In the fetch stage rows are selected and ordered.

Q-75 Explain database structure?

----Physical structure

- => One or more datafiles
- => Two or more redolog files (one for reading & another for writing)
- => One or more control files.

----Logical structure

- => Table spaces
- => Schema objects
- => Segments – collection of extents.
- => Extents – collection of data blocks.
- => Datablocks

Q-76 Explain the physical structure of database?

----Data files

Entire data of database is stored in the datafiles forms a logical unit called as tablespace.

----Redolog files

Stores the log of changes made to the data of the database. collection of redolog files is called redlog.

----Control files

Stores name and timestamp of DB creation.

Q-77 Explain the logical structure of DB?

----Data Blocks

It is the smallest unit of I/O operation used by oracle to read or write data on hard disk.

----Extents

It is collection of datablocks.

----Segment

It is a collection of multiple extents.

There are four types of segments

- ⇒ Data segment
- ⇒ Index segment
- ⇒ Rollback segment
- ⇒ Temp segment

----Table space

A tablespace is a collection of schema objects. One tablespace can contains objects of multiple schemas.

Let's have some discussion about long running query in oracle

There is no absolute or concrete answer to this question because there might be multiple way to tune a slow running query. Everyone can have a different approach.

So here is my approach:

Very first thing we have to find which type of query is running slow. Whether it is SELECT query or DML statements(insert, update, delete and merge).

So first we will see how to tune a SELECT query

Step 1 – Find the SQL_ID of the slow running query

There could be two possibilities:

1) Query is still running: If the slow query is still running then we can find the sql_id of the query by using v\$sqlsession view.

```
SQL>select sql_id
from v$sqlsession
where sid = 3089;
```

Output:

```
SQL_ID
-----
g0uubmuvk4uax
```

Note : The SID is a site identifier (where query is running [database identifier])
How to find the sid -- select instance from v\$thread

2) Query is completed: It might be query is completed but application team got to know it later that the query was slow and it did not finish in its usual time. Since this query is not running right now, we can't query v\$sqlsession to get its information.

So for this we use AWR/ASH report. We need to tell DBA team that at what time the query was running and for that duration DBA generate the AWR [**Automatic Workload Repository**]. In AWR report we find all kind of information regarding the top SQL's. Basically we see SQL STATISTICS section of the AWR report. If the query which is reported by application team is present in this section then DBA note down the sql_id of the query. Otherwise DBA generate ASH [**Active Session History**] report for that particular time and get the sql_id.

The ASH contains recent information on active sessions sampled every second (insert from v\$sqlsession to V\$ACTIVE_SESSION_HISTORY)

The AWR are taken every one hour.

AWR -- stores the session performance statistics for analysis later.

AWR gathers the following data

- ❖ Object Statistics
- ❖ Time Model Statistics
- ❖ Some of the System and Session Statistics
- ❖ ASH (Active Session History) Statistics
- ❖ High load generating SQL Statements

AWR gives you only top SQLs (gives Big picture for particular time period and doesn't contain all the SQLs)

ASH -- The old(completed/inactive) entries are removed to accommodate new ones. Old(completed) SQLs can be viewed using V\$ACTIVE_SESSION_HISTORY.

Above view also contains data of last 60 mins only and after 60 mins DBA can view the details from below view.

DBA_HIST_ACTIVE_SESSION_HISTORY

Above view doesn't take all the information , it just take 1 in 10 secs means only SQLs which ran more than 10 secs (v\$session takes snapshot every seconds).

Step 2 – Run the SQL Tuning advisor for that SQL_ID

After finding the sql_id we can run sql tuning advisor on this sql_id.

The **DBMS_SQLTUNE** package provides the interface to tune SQL statements.

Use below steps to run the sql tuning advisor and display its output:

1. Create tuning task for specific Sql id:

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
l_sql_tune_task_id VARCHAR2(100);
```

```
BEGIN
```

```
l_sql_tune_task_id := DBMS_SQLTUNE.create_tuning_task (
    sql_id    => 'g0uubmuvk4uax',
    scope     => DBMS_SQLTUNE.scope_comprehensive,
    time_limit => 60,
    task_name => 'g0uubmuvk4uax_tuning_task',
    description => 'Tuning task for statement g0uubmuvk4uax.');
```

```
DBMS_OUTPUT.put_line('l_sql_tune_task_id: ' || l_sql_tune_task_id);
```

```
END;
```

```
/
```

2. Execute the tuning task:

```
EXEC DBMS_SQLTUNE.execute_tuning_task(task_name => 'g0uubmuvk4uax_tuning_task');
```

3. Display the recommendations:

Once the tuning task has executed successfully the recommendations can be displayed using the REPORT_TUNING_TASK function.

```
SET LONG 10000;
```

```
SET PAGESIZE 1000
```

```
SET LINESIZE 200
```

```
SELECT DBMS_SQLTUNE.report_tuning_task('g0uubmuvk4uax_tuning_task') AS recommendations  
FROM dual;
```

```
SET PAGESIZE 24
```

Based on the tuning advisor recommendation we have to take corrective actions. These recommendation could be one and many more:

- 1) Gather Statistics [execution plan]*
- 2) Create Index*
- 3) Drop Index*
- 4) Join orders*
- 5) Accept sql profile*
- 6) Create baseline and many more recommendations*

Before applying any corrective action on production, we need to test that in DEV/QA/UAT/TEST environment or we can ask an tuning expert that this is the recommendation. After all the analysis we should apply in the production database.

After corrective action from tuning advisor run the SQL again and see the improvement.

- **What is meant by gathering the stats on a table**

If I have to buy 2 things at the shop, I might walk there.
If I have to buy 10 things, I might walk there and take a bag with me.
If I have to buy 200 items...I'll probably drive my car.

Hence the **size** of a task, often determines **how** I will carry out that task.

The database is the same. It records information on the size of tables so it can best work out how to run queries. This information is called "statistics" and hence we "gather" those statistics using the DBMS_STATS package.

Step 3 - Check the sql plan hash value and pin the good plan:

Sometime SQL tuning advisor does not recommend anything, in that case we have to go for a different approach.

check if there is any change in the sql plan.

We can also use below query to find out the sql plan for n number of days:
(Show the plan has value for a given sqlid over a given period)

```
SET PAUSE ON
```

```
SET PAUSE 'Press Return to Continue'
```

```
SET PAGESIZE 60
SET LINESIZE 300
```

```
SELECT DISTINCT sql_id, plan_hash_value
FROM dba_hist_sqlstat q,
(
  SELECT /*+ NO_MERGE */ MIN(snap_id) min_snap, MAX(snap_id) max_snap
  FROM dba_hist_snapshot ss
  WHERE ss.begin_interval_time BETWEEN (SYSDATE - &No_Days) AND SYSDATE
) s
WHERE q.snap_id BETWEEN s.min_snap AND s.max_snap
AND q.sql_id IN ( '&SQLID')
/
```

here we have to supply the value for number of days.

We can see the entire plan for n number of days the query has used. It is not mandatory that plan is changed. Here it also displays which plan takes how much time executes. So if there is change in plan, we should find best plan and pin it. Plan could be changed due to several reasons like change in the sql, due to stats gathering and many more.

If it is a DML statement then we have check the locking in the database [also increase in data size could be major reason]

[Step 4 - Find the locking in the database:](#)

The very first step is to find out if there is any locking in the database. Sometime due to locking a session does not get the required resources and the session gets slow.

We can find below command to check locking in the database:

In Standalone:

```
sql>select s1.username || '@' || s1.machine || ' ( SID=' || s1.sid || ' ) is blocking'
|| s2.username || '@' || s2.machine || ' ( SID=' || s2.sid || ' )' from
v$sql l1, v$sqlsession s1, v$sql l2, v$sqlsession s2
where s1.sid=l1.sid and s2.sid=l2.sid
and l1.block=1 and l2.request > 0
and l1.id1=l2.id1
and l2.id2=l2.id2;
```

In RAC:

```
sql> select s1.username || '@' || s1.machine || ' ( SID=' || s1.sid || ' ) is blocking'
|| s2.username || '@' || s2.machine || ' ( SID=' || s2.sid || ' )' from
gv$sql l1, gv$sqlsession s1, gv$sql l2, gv$sqlsession s2
```

```

where s1.sid=l1.sid and s2.sid=l2.sid
and l1.block=1 and l2.request > 0
and l1.id1=l2.id1
and l2.id2=l2.id2;

```

Query Output:

```

S1.USERNAME||'@'||S1.MACHINE||'(SID='||S1.SID||')ISBLOCKING'||S2.USERNAME||'@'||
-----
SYS@host1.abc.com ( SID=229) is blockingSYS@host1.abc.com( SID=226)

```

If we see the locking from above query then we can simply inform to user/application. And if they suggest to kill this blocking session then after killing we can get rid of this slowness.

Step 6 - Check for the wait events:

There could be some wait events on the database. Check for the particular user and session.

Query for displaying sessions, session state, and wait details

col "Description" format a50

```

select sid,
       decode(state, 'WAITING','Waiting',
              'Working') state,
       decode(state,
              'WAITING',
              'So far '||seconds_in_wait,
              'Last waited '||
              wait_time/100)||
              ' secs for '||event
       "Description"

```

from v\$session

where username = 'ARUP';

Output:

```

SID  STATE  Description
-----

```

```

-----
-----

```

```

2832 Working  Last waited 2029 secs for SQL*Net message from client
3346 Waiting  So far 743 secs for enq: TX - row lock contention
4208 Waiting  So far 5498 secs for SQL*Net message from client

```


It clearly shows the state of the sessions: whether they are working or waiting; if they are working, what they were waiting for earlier and for how long; and if they are waiting, what for and for how long.

In many troubleshooting situations, just knowing the SID of each session is not enough. We may need to know other details, such as the client machine the session is connecting from, the user (of both the database and the operating system), and the service name. All of this information is also readily available in the same V\$SESSION view we have been using. Let's briefly examine the columns that provide that information, by running the below query

```
select SID, osuser, machine, terminal, service_name,
       logon_time, last_call_et
from v$session
where username = 'ARUP';
```

```
SID  OSUSER  MACHINE  TERMINAL  SERVICE_NAME  LOGON_TIME  LAST_CALL_ET
```

```
-----
```

```
-----
```

```
-----
```

```
3346 oradb  prodb1  pts/5    SYS$USERS   05-FEB-12    6848
```

```
2832 oradb  prodb1  pts/6    SERV1       05-FEB-12    7616
```

```
4408 ANANDA  ANLAP   ANLAP    ADHOC       05-FEB-12     0
```

Step 7 - Getting the SQL:

Another key piece of performance tuning information is the SQL statement a session is executing, which will provide more insights into the workings of the session. The same V\$SESSION view also shows the SQL statement information. The SQL_ID column in the V\$SESSION view shows the ID of the last SQL statement executed. We can get the text of that SQL statement from the V\$SQL view, using the SQL_ID value. Here is an example of how I have identified the SQL statement executed by the session that appears slow to the user.

```
select sql_id
from v$session
where sid = 3089;
```

```
SQL_ID
```

```
-----
```

```
g0uubmuvk4uax
```

```
set long 99999
select sql_fulltext
```

```

from v$sql
where sql_id = 'g0uubmuvk4uax';
SQL_FULLTEXT

```

```

update t1 set col2 = 'y' where col1 = 1

```

Step 8 - Data Access issue:

Although locking-related contention is a very common cause, it is not the only cause of performance problems. Another major cause of contention is disk I/O. When a session retrieves data from the database data files on disk to the buffer cache, it has to wait until the disk sends the data. This wait shows up for that session as “db file sequential read” (for index scans) or “db file scattered read” (for full-table scans) in the EVENT column, as shown below:

```

select event
from v$session
where sid = 3011;

```

```

EVENT

```

```

db file sequential read

```

When we see this event, we know that the session is waiting for I/O from the disk to complete. To make the session go faster, we have to reduce that waiting period. There are several ways to reduce the wait:

Reduce the number of blocks retrieved by the SQL statement. Examine the SQL statement to see if it is doing a full-table scan when it should be using an index, if it is using a wrong index, or if it can be rewritten to reduce the amount of data it retrieves.

Place the tables used in the SQL statement on a faster part of the disk.

Consider increasing the buffer cache to see if the expanded size will accommodate the additional blocks, therefore reducing the I/O and the wait.

Tune the I/O subsystem to return data faster.

SQL Performance Questions:

Q:-1. What is SQL Query Optimization?

Ans. Query Optimization is the process of writing the query in a way so that it could execute quickly. It is a significant step for any standard application.

Q:-2. What are some tips to improve the performance of SQL queries?

Ans. Optimizing SQL queries can bring substantial positive impact on the performance. Let's now go over some of the tips for tuning SQL queries.

1. **Prefer to use views and stored procedures in spite of writing long queries.** It'll also help in minimizing network load.
2. **It's better to introduce constraints instead of triggers.** They are more efficient than triggers and can increase performance.
3. Make use of table-level variables instead of temporary tables.
4. **The UNION ALL clause responds faster than UNION.** It doesn't look for duplicate rows whereas the UNION statement does that regardless of whether they exist or not.
5. **Prevent the usage of DISTINCT and HAVING clauses.**
6. **Avoid excessive use of SQL cursors.**
7. Make use of SET NOCOUNT ON clause while building stored procedures. It represents the rows affected by a T-SQL statement. It would lead to reduced network traffic.
8. It's a good practice to return the required column instead of all the columns of a table.
9. Prefer not to use complex joins and avoid disproportionate use of triggers.
10. **Create indexes for tables** and adhere to the standards.

Q:-3. What are the bottlenecks that affect the performance of a Database?

Ans. In a web application, the database tier can prove to be a critical bottleneck in achieving the last mile of scalability. If a database has performance leakage, that can become a bottleneck and likely to cause the issue. Some of the common performance issues are as follows.

1. Abnormal CPU usage is the most obvious performance bottleneck. However, you can fix it by extending CPU units or replacing with an advanced CPU. It may look like a simple issue but abnormal CPU usage can lead to other problems.
2. Low memory is the next most common bottleneck. If the server isn't able to manage the peak load, then it poses a big question mark on the performance. For any application, memory is very critical to perform as it's way faster than the persistent memory. Also, when the RAM goes down to a specific threshold, then the OS turns to utilize the swap memory. But it makes the application to run very slow.

You can resolve it by expanding the physical RAM, but it won't solve memory leaks if there is any. In such a case, you need to profile the application to identify the potential leaks within its code.

3. Too much dependency on external storage like SATA disk could also come as a bottleneck. Its impact gets visible while writing large data to the disk. If output operations are very slow, then it is a clear indication an issue becoming the bottleneck.

In such cases, you need to do scaling. Replace the existing drive with a faster one. Try upgrading to an SSD hard drive or something similar.

Q:-4. What are the steps involved in improving the SQL performance?

Ans. Discover – First of all, find out the areas of improvement. Explore tools like Profiler, Query execution plans, SQL tuning advisor, dynamic views, and custom stored procedures.

Review – Brainstorm the data available to isolate the main issues.

Propose – Here is a standard approach one can adapt to boost the performance. However, you can customize it further to maximize the benefits.

1. Identify fields and create indexes.
2. Modify large queries to make use of indexes created.
3. Refresh table and views and update statistics.
4. Reset existing indexes and remove unused ones.
5. Look for dead blocks and remove them.

Validate – Test the SQL performance tuning approach. Monitor the progress at a regular interval.

Also, track if there is any adverse impact on other parts of the application.

Publish – Now, it's time to share the working solution with everyone in the team. Let them know all the best practices so that they can use it with ease.

Q:-5. What is a explain plan?

Ans. It's a term used in Oracle. And it is a type of SQL clause in Oracle which displays the execution plan that its optimizer plans for executing the SELECT/UPDATE/INSERT/DELETE statements.

Q:-6. How do you analyse an explain plan?

Ans. While analysing the explain plan, check the following areas.

1. Driving Table
2. Join Order
3. Join Method
4. Unintentional cartesian product
5. Nested loops, merge sort, and hash join
6. Full Table Scan
7. Unused indexes
8. Access paths

Q:-7. How do you tune a query using the explain plan?

Ans. The explain plan shows a complete output of the query costs including each subquery. The cost is directly proportional to the query execution time. The plan also depicts the problem in queries or sub-queries while fetching data from the query.

Q:-8. What is Summary advisor and what type of information does it provide?

Ans. Summary advisor is a tool for filtering and materializing the views. It can help in elevating the SQL performance by selecting the proper set of materialized views for a given workload. And it also provides data about the Materialized view recommendations.

Q:-9. What could most likely cause a SQL query to run as slow as 5 minutes?

Ans. Most probably, a sudden surge in **the volume of data in a particular table** could slow down the output of a SQL query. So collect the required stats for the target table. Also, monitor any change in the DB level or within the underlying object level.

Q:-10. What is a Latch Free Event? And when does it occur? Also, how does the system handles it?

Ans. In Oracle, Latch Free wait event occurs when a session requires a latch, attempts to get it but fails because someone else has it.

So it sleeps with a wait eyeing for the latch to get free, wakes up and tries again. The time duration for it was inactive is the wait time for Latch Free. Also, there is no ordered queue for the waiters on a latch, so the one who comes first gets it.

Q:-11. What is Proactive tuning and Reactive tuning?

Ans. Proactive tuning – The architect or the DBA determines which combination of system resources and available Oracle features fulfil the criteria during Design and Development.

Reactive tuning – It is the bottom-up approach to discover and eliminate the bottlenecks. The objective is to make Oracle respond faster.

Q:-12. What are Rule-based Optimizer and Cost-based Optimizer?

Ans. Oracle determines how to get the required data for processing a valid SQL statement. It uses one of following two methods to take this decision.

Rule-based Optimizer – When a server doesn't have internal statistics supporting the objects referenced by the statement, then the RBO method gets preference. However, Oracle will deprecate this method in the future releases.

Cost-based Optimizer – When there is an abundance of the internal statistics, the CBO gets the precedence. It verifies several possible execution plans and chooses the one with the lowest cost based on the system resources.

Q:-13. What are several SQL performance tuning enhancements in Oracle?

Ans. Oracle provides many performance enhancements, some of them are:

1. Automatic Performance Diagnostic and Tuning Features
2. Automatic Shared Memory Management – It gives Oracle control of allocating memory within the SGA.
3. Wait-model improvements – A number of views have come to boost the Wait-model.

4. Automatic Optimizer Statistics Collection – Collects optimizer statistics using a scheduled job called GATHER_STATS_JOB.
5. Dynamic Sampling – Enables the server to enhance performance.
6. CPU Costing – It's the basic cost model for the optimizer (CPU+I/O), with the cost unit as time optimizer notifies.
7. Rule Based Optimizer Obsolescence – No more used.
8. Tracing Enhancements – End to End tracing which allows a client process to be identified via the Client Identifier instead of using the typical Session ID.

Q:-14. What are the tuning indicators Oracle proposes?

Ans. The following high-level tuning indicators are available to establish if a database is experiencing bottlenecks or not:

1. Buffer Cache Hit Ratio.

It uses the following formula.

Hit Ratio = (Logical Reads – Physical Reads) / Logical Reads

Action: Advance the DB_CACHE_SIZE (DB_BLOCK_BUFFERS prior to 9i) to improve the hit ratio.

2. Library Cache Hit Ratio.

Action: Advance the SHARED_POOL_SIZE to increase the hit ratio.

Q:-15. What do you check first if there are multiple fragments in the SYSTEM tablespace?

Ans. First of all, check if the users don't have the SYSTEM tablespace as their TEMPORARY or DEFAULT tablespace assignment by verifying the DBA_USERS view.

Q:-16. When would you add more Copy Latches? What are the parameters that control the Copy Latches?

Ans. If there is excessive contention for the Copy Latches, check from the "redo copy" latch hit ratio.

In such a case, add more Copy Latches via the initialization parameter LOG_SIMULTANEOUS_COPIES to double the number of CPUs available.

Q:-17. How do you confirm if a tablespace has disproportionate fragmentation?

Ans. You can confirm it by checking the output of SELECT against the dba_free_space table. If it points that the no. of a tablespaces extents is more than the count of its data files, then it proves excessive fragmentation.

Q:-18. What can you do to optimize the %XYZ% queries?

Ans. First of all, set the optimizer to scan all the entries from the index instead of the table. You can achieve it by specifying hints.

Please note that crawling the smaller index takes less time than to scan the entire table.

Q:-19. Where does the I/O statistics per table exist in Oracle?

Ans. There is a report known as UTLESTAT which displays the I/O per tablespace. But it doesn't help to find the table which has the most I/O.

Q:-20. When is the right time to rebuild an index?

Ans. First of all, select the target index and run the 'ANALYZE INDEX VALIDATE STRUCTURE' command. Every time you run it, a single row will get created in the INDEX_STATS view.

But the row gets overwritten the next time you run the ANALYZE INDEX command. So better move the contents of the view to a local table. Thereafter, analyse the ratio of 'DEL_LF_ROWS' to 'LF_ROWS' and see if you need to rebuild the index.

Q:-21. What exactly would you do to check the performance issue of SQL queries?

Ans. Mostly the database isn't slow, but it's the worker session which drags the performance. And it's the abnormal session accesses which cause the bottlenecks.

1. Review the events that are in wait or listening mode.
2. Hunt down the locked objects in a particular session.
3. Check if the SQL query is pointing to the right index or not.
4. **Launch SQL Tuning Advisor and analyse the target SQL_ID for making any performance recommendation.**
5. Run the "free" command to check the RAM usage. Also, use TOP command to identify any process hogging the CPU.

Q:-22. What is the information you get from the STATSPACK Report?

Ans. We can get the following statistics from the STATSPACK report.

1. WAIT notifiers
2. Load profile
3. Instance Efficiency Hit Ratio
4. Latch Waits
5. Top SQL
6. Instance Action
7. File I/O and Segment Stats
8. Memory allocation
9. Buffer Waits

Q:-23. What are the factors to consider for creating Index on Table? Also, How to select a column for Index?

Ans. Creation of index depends on the following factors.

1. Size of table,
2. Volume of data

If Table size is large and we need a smaller report, then it's better to create Index.

Regarding the column to be used for Index, as per the business rule, you should use a primary key or a unique key for creating a unique index.

Q:-24. What is the main difference between Redo, Rollback, and Undo?

Ans. Redo – Log that records all changes made to data, including both uncommitted and committed changes.

Rollback – Segments to store the previous state of data before the changes.

Undo – Helpful in building a read consistent view of data. The data gets stored in the undo tablespace.

Q:-25. How do you identify the shared memory and semaphores of a particular DB instance if there are running multiple servers?

Ans. Set the following parameters to distinguish between the in-memory resources of a DB instance.

1. SETMYPID

2. IPC

3. TRACEFILE_NAME

Use the ORADEBUG command to explore their underlying options.

SQL Performance Tuning Questions:

1.What are different parameters to consider the database performance of Application?

Ans: There are lot of parameters to consider the performance of application:

a) What size of images we are using in application. The images we are using on application should not be maximum size.

b) What is the **data volume used to fetch** the data

c) Data cardinality: The most important factor is data cardinality of the data in application. **Data should be divided in proper manner and the database should be in well normalized form**

d) **Indexing done**: Indexing should be done properly in database.

2.What are indexes in SQL?

Ans: Index is optional structure associated with the table which may or may not improve the performance of Query.

In simple words suppose we want to search the topic in to book we go to index page of that book and search the topic which we want. Just like that to search the values from the table when indexing is there you need not use the full table scan.

3.What are advantages of Indexes?

Ans: Indexes are memory objects which are used to improve the performance of queries which allows faster retrieval of records.

Following are advantages of Indexes:

a) It allows faster retrieval of data

b) It avoids the Full table scan so that the performance of retrieving data from the table is faster.

c) It avoids the table access altogether

d) Indexes always speeds up the select statement.

e) Indexes used to improve the Execution plan of the database

4.What are disadvantages of Indexes?

Ans: a) Indexes slows down the performance of insert and update statements. So always we need follow best practice of disabling indexes before insert and update the table

b) Indexes takes additional disk space so by considering memory point indexes are costly.

5.What is parser?

Ans: When SQL Statement has been written and generated then first step is parsing of that SQL Statement. Parsing is **nothing but checking the syntaxes of SQL query**. All the syntax of Query is correct or not is checked by SQL Parser.

There are 2 functions of parser:

a) Syntax analysis

b) Semantic analysis

6.What are functions of Parser?

Ans: a) Syntax Analysis: The parser checks for SQL statement syntaxes .If the syntax is incorrect then parser gives the incorrect syntax error.

b) Semantic Analysis: This checks for references of object and object attributes referenced are correct.

7.What is mean by implicit index?

Ans: Whenever we define unique key or primary key constraints on the table the index will automatically create on the table. These indexes are known as implicit indexes because these are created implicitly whenever the constraint has been applied to the table. These indexes are normal indexes not unique indexes. The indexes are normal because the columns already have defined as unique so uniqueness is already been applied.

8.What are Explicit Indexes?

Ans: The indexes which is created by user are called as explicit indexes. You can say the indexes which are created by 'Create Index' statement are called as Explicit indexes.

Syntax:

```
create index <index_name> on tablename(column_name);
```

9.What are different types of indexes?

Ans: There are following types of indexes:

a) Normal Indexes

b) Bit map indexes

- c) B-tree Indexes
- d) Unique Indexes
- e) Function Based Indexes

10.What is mean by Unique Indexes?

Ans: a) To create unique index you must have CREATE ANY INDEX privilege. Here the concept is bit different. User needs to check the values of the table to create unique index. If table contains uniquely identified values in specified column then you should use unique index.

b) Especially while creating the table if we specify the primary key then unique index is automatically created on that column.

c) But for Unique key constraint columns you separately need to do indexing. Kindly make sure that Unique key indexes created on the columns which has unique values only.

d) The unique indexes are also called as clustered indexes when primary key is defined on the column.

Example:

Create Unique index Index_name on Table_name(Unique column name);

11.What are functional Based indexes?

Ans: a) Function based indexes allows us to index on the functional columns so that oracle engine will take the index and improves the performance of the query.

b) As per requirements we are using lot of SQL functions to fetch the results. Function based indexes gives ability to index the computed columns.

c) Function based indexes are easy to implement and it also provides immediate value. These indexes speeds up the application without changing application code or query.

Syntax:

Create index indexname on tablename(Function_name(column_name));

Example:

Create index FI_Employee on Employee(trunc(Hire_date));

12.What is Bit-map index?

Ans: a) If Table contains the distinct values which are not more than 20 distinct values then user should go for Bit map indexes.

b) User should avoid the indexing on each and every row and do the indexing only on distinct records of the table column. You should able to check drastic change in query cost after changing the normal index to Bit map index.

c) The bit map indexes are very much useful in data ware housing where there are low level of concurrent transactions. Bit map index stores row_id as associated key value with bitmap and did the indexing only distinct values.

d) Means if in 1 million records only 20 distinct values are there so Bitmap index only stores 20 values as bitmap and fetches the records from that 20 values only.

Syntax:

Create bitmap index Index_name on Table_name(Columns which have distinct values);

14.What is B-tree index?

Ans: A b-tree index works by creating a series of nodes in a hierarchy. It's often compared to a tree, which has a root, several branches, and many leaves.

For example if you want to find a record with an ID of 109

The steps to find this record would be:

- a) Start at the root node and go to the first level.
- b) Find the node that covers the range of values that span the value of 109 (for example, a range of 100 to 200).
- c) Move to the second level from this node.
- d) Find the node that covers the range of values that span the value of 109 on the second level (for example, 100 to 120).
- e) Move to the third level from this node.
- d) Find the record that has an ID of 109.

15.What is Optimizer?

Ans: Optimizer is nothing but the execution of query in optimum manner. Optimizer is most efficient way of processing the query. SQL parser ,SQL Optimizer and source code generator compiles the SQL statement.

16.What are types of SQL Optimizer?

Ans: There are following types of optimizer:

- a) Rule Based Optimizer
- b) Cost Based Optimizer

17.Explain Rule Based Optimizer?

Ans: When we execute any SQL statement ,the optimizer uses the predefined rules which defines what indexes are present in the database and which indexes needs to be executed during the execution. Rule Based optimizer is used to specify which table is been full scanned and which tables are taking the indexes during the execution. In Earlier the only optimizer which is used by Oracle is Rule Based optimizer

“Rule Based Optimizer specifies the rules for how to execute the query.”

18.What is composite index?

Ans: When 2 or more columns are related to each other in the table and the same columns are used in where condition of the query then user can create index on both columns. These indexes are known as composite indexes.

Example:

Create index CI_Employee on Employee(Eno,Deptno);

19.What is cost based optimizer?

Ans: Cost Based Optimizer (CBO) uses the artificial intelligence to execute the Query. The Optimizer itself decides the execution plan based on the cost of query. The cost based method means the database must decide which query execution plan to choose. It decides the query execution plan by deciding the statistical information based on the database objects.(tables, indexes and clusters).The Oracle cost based optimizer is designed to determine the most effective way to carry out the SQL statement.

“Cost based optimizer considers the statistical information of the table for query execution”

20.What is mean by Clustered index?

Ans: a) The clustered indexes are indexes which are physically stored in order means it stores in ascending or descending order in Database.
b) Clustered indexes are created once for each table. When primary key is created then clustered index has been automatically created in the table.
c) If table is under heavy data modifications the clustered indexes are preferable to use
d) Cluster Index is as same as dictionary where the data is arranged by alphabetical order.

21.Can Index be Renamed? If Yes How?

Ans: Yes we can rename the indexes. User should have create any index privilege to rename the index.

Alter index Index_name Rename to New_indexname;

22.What is mean by non-clustered indexes?

Ans: a) The clustered indexes are used for searching purpose as we can create clustered indexes where primary is defined. But Non clustered indexes are indexes which will be created on the multiple joining conditions, multiple filters used in query.
b) We can create 0 to 249 non-clustered indexes on single table. Foreign keys should be non-clustered.
c) When user wants to retrieve heavy data from fields other than primary key the non-clustered indexes are useful.
d) A non-clustered index is as same as to an index of a book.
e) Data is stored in one place and index is stored in another place.

23. What is merge statement in SQL ?

Suppose, you have two table called source and target tables, and you need to update the target table based on the values matched from the source table. There are three cases:

- 1) The source table has some rows that do not exist in the target table. In this case, you need to insert rows that are in the source table into the target table.
- 2) The target table has some rows that do not exist in the source table. In this case, you need to delete rows from the target table.
- 3) The source table has some rows with the same keys as the rows in the target table. However, these rows have different values in the non-key columns. In this case, you need to update the rows in the target table with the values coming from the source table.

```
MERGE target_table USING source_table
ON merge_condition
WHEN MATCHED
    THEN update_statement
WHEN NOT MATCHED
    THEN insert_statement
WHEN NOT MATCHED BY SOURCE
    THEN DELETE;
```

```
MERGE category c USING category_staging s
ON (s.category_id = c.category_id)
WHEN MATCHED
    THEN UPDATE SET
        c.category_name = s.category_name,
        c.amount = s.amount
WHEN NOT MATCHED BY TARGET
    THEN INSERT (category_id, category_name, amount)
        VALUES (s.category_id, s.category_name, s.amount)
WHEN NOT MATCHED BY SOURCE
    THEN DELETE;
```